



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24403

学 号： 24068240317

姓 名： 刘远鹏

实验（1）

【实验目的】

1. 掌握基础提示词的构成要素：深入理解提示词（Prompt）的四大核心要素——指令（Instruction）、上下文（Context）、输入数据（Input）和输出指示器（Output Indicator），并能独立构建基础提示词。
2. 对比“模糊指令”与“明确指令”的差异：通过对比实验，直观验证指令的清晰度、具体性对模型输出质量的决定性影响，理解“垃圾进，垃圾出”（Garbage In, Garbage Out）在 AI 交互中的体现。
3. 建立规范的交互思维：学习如何将自然语言转化为机器可理解的逻辑指令，避免歧义，确保模型能够准确捕捉用户意图并完成指定任务（如信息总结、文本生成、简单问答）。

【实验内容】

1.2 实验过程：

使用 AI 模型完成信息总结、文本生成、简单问答三类基础任务。

1.2 实验结果：

只会手写代码真的会被淘汰吗

MiMo-V2-Pro

已深度思考 (用时 5.3 秒) ▾

这个问题值得认真聊聊，因为它比表面看起来要复杂。

"手写代码"这个概念需要拆开

会变的部分：

- 逐行敲语法、查 API 文档、写样板代码 — 这些确实会被 AI 大幅替代
- 纯粹靠“记住怎么写”的能力，价值在下降

不会变的部分：

- 把模糊的业务需求拆解成清晰的逻辑结构
- 判断一个方案在高并发、大数据量下会不会崩
- 调试一个诡异的线上问题，理解系统各部分怎么咬合
- 做架构权衡 — 一致性还是可用性，耦合还是复用

更准确的说法是

被淘汰的不是“手写代码的人”，而是只会做翻译的人 — 把需求文档机械地翻译成代码，中间没有自己的思考和判断。

AI 能生成代码，但它目前很难：

- 判断这段代码该不该写
- 理解上下文约束（性能、安全、团队维护成本）
- 在多个可行方案中做取舍

一个类比

计算器没有淘汰数学家，反而让数学家把精力放在更高层次的问题上。AI 编程工具也是类似的角色 — 它降低的是执行层的门槛，但对理解和判断力的要求反而更高了。

所以与其担心“手写代码”会不会被淘汰，不如问自己：你对系统的理解，是停留在“能跑就行”，还是能讲清楚为什么这样设计？

能回答后一个问题的人，不管工具怎么变，都不会被淘汰。

【实验总结】

从模糊到精准： 实验初期，我使用了较为随意的自然语言提问，结果往往宽泛且缺乏重点。通过引入“明确任务+限定条件+规定格式”的结构化公式，模型的输出质量显著提升。例如，在信息总结任务中，明确要求“用三点概括”比“总结一下”能得到更

条理清晰的回答。

基础的重要性： 我理解了基础提示词是后续高级技巧的基石。一个高质量的输出，首先依赖于指令的完整性。模糊的指令会导致模型产生“幻觉”或偏离主题，而规范的提示能极大提升任务执行的效率和准确率。

反思： 后续需要继续强化对任务需求的拆解能力，学会用最精炼的语言传达最核心的意图。

实验（2）

【实验目的】

1. 掌握高级提示工程技巧： 熟练运用角色设定（Role Playing）、思维链（Chain of Thought）、少样本学习（Few-Shot Learning）以及上下文补充等高级策略，引导模型模拟特定领域的专家思维。
2. 提升输出的专业性与一致性： 通过引入背景信息、使用场景及分步指令，验证这些技巧在解决复杂问题时对模型输出逻辑性、专业性和实用性的提升效果。
3. 构建复杂的交互逻辑： 学习如何通过示例引导（Example 引导）和约束条件，控制模型的语气、风格和格式，使其输出更符合特定的业务或学术场景需求。

【实验内容】

2.1 实验过程：



1. 为每个任务构建高级提示词：

- 加入角色身份（如：AI 工程师）
- 补充背景信息与使用场景
- 使用分步指令与示例输出
- 规定输出格式与约束条件

2. 运行模型并记录输出结果，评估专业性、准确性、完成度。

2.2 实验结果：

现在你是一个AI超级工程师，会ai编码，会训练AI。我是一名手写的代码的工程师，只会手写代码真的会被淘汰吗

MiMo-V2-Pro

已深度思考 (用时 0.6 秒) ▾

这个问题你问对人了。我每天都在用 AI 写代码、训练模型，对它的能力边界很清楚。跟你说实话。

AI 编码现在到底什么水平

它能做的：

- 写 CRUD、写接口、写样板代码 — 又快又好
- 读代码、解释代码、改 bug — 大部分场景够用
- 根据描述生成完整的小项目 — 脚手架级别的活儿基本能干
- 写测试用例、文档、迁移脚本 — 效率提升巨大

它做不好的：

- 复杂系统的架构决策 — 它没有全局视角，不知道你的系统跑在什么环境、服务多少用户、有哪些历史包袱
- 性能调优 — 它能给建议，但真正的瓶颈在哪，需要人去 profiling、去分析
- 跨系统的集成 — 各种脏数据、协议差异、边界情况，AI 经常编造它不确定的东西
- 安全性判断 — 它生成的代码可能有漏洞，它自己不知道

你最该担心什么

不是“AI 会写代码”这件事本身。

真正该担心的是：一个会用 AI 工具的工程师，效率可能是你的 3-5 倍。

这不是说你被淘汰了，而是竞争格局变了。就像：

- 以前手绘设计稿的设计师，Photoshop 出来后没被淘汰，但不会用的确实掉队了
- 以前纯手算的会计，Excel 出来后没被淘汰，但只会算盘的确边缘化了

我作为用 AI 写代码的人，跟你讲几个真相 ▾

【实验总结】

技巧的威力： 实验中，单纯的角色设定（如“你是一位资深

AI 工程师”）就让回答的深度有了质的飞跃。结合“分步思考”的指令，模型在处理复杂逻辑问题时不再跳跃，而是呈现出清晰的推理链条。

结构化引导：我发现，通过提供“示例引导”（Few-Shot），模型能快速模仿特定的格式和风格。这证明了“示范”比单纯的“说教”在 AI 交互中更有效。

结论：高级提示技巧能显著降低模型产生幻觉的概率，并大幅提升其在专业场景下的可用性。提示工程不是简单的对话，而是利用结构化语言引导模型精准输出的艺术。

实验（3）

【实验目的】

1. 综合运用全流程：结合前两次实验的知识，针对一个完整的、复杂的场景化任务，设计并实施一套完整的提示工程方案。
2. 掌握调试与迭代方法：学习如何分析模型输出中的问题（如信息遗漏、逻辑混乱、格式错误），并针对性地通过补充约束、简化指令、强化结构等方式进行迭代优化。
3. 建立工程化思维：形成一套标准化的“设计—测试—修正—定稿”的提示工程 workflow，培养解决实际复杂问题的能力

【实验内容】

3.1 实验过程：

1. 编写初版提示词，提交模型获取输出。
2. 分析问题：如信息遗漏、逻辑混乱、格式错误、语气不符等。
3. 针对性优化提示词：补充约束、简化指令、强化结构、增加示例。
4. 多次迭代，直到输出稳定、可用、高质量，记录每版差异。
5. 整理最终提示词与最优输出，形成实验成果。

3.2 实验结果：

请帮我生成一本以都市高武的爽文，剧情丰富完整，章节在50章左右

没问题！都市高武题材的核心就在于“扮猪吃虎”与“战力碾压”的极致快感。

根据你的要求，我为你策划了这本设定严谨、节奏紧凑的50章爽文大纲。故事主打“系统流+身份反转”，目前已经规划好了从废柴开局到揭开世界真相的完整脉络：

书名：《都市：万倍返还，从武道通神开始》

核心设定

- 世界观：现代都市与武道修行并存，全球灵气复苏百年，武者是特权阶级。人类依靠气血值修炼，对抗深渊异兽。
- 核心爽点：只要被轻视/嘲讽，主角的修炼速度、悟性将获得万倍返还。
- 人物档案：林默，青城大学大三学生，表面是气血值仅0.5的“废柴”，实则是隐藏的“满级大佬”。
- 创作亮点：利用“信息差”制造爽感，别人眼中的绝世凶兽，在主角眼里只是经验宝宝。

第一卷：蛰伏的巨龙

第一章：史上最废气血值

- 情节：青城大学体测中心，大屏幕显示林默气血值0.5（正常成年男性应>10）。周围同学哄笑，女友苏雨柔当众提出分手，投入武道天才赵天宇怀抱。系统激活：【检测到恶意轻视，悟性返还倍率开启】。
- 关键点：极致的压抑，铺垫主角“扮猪”的必要性。

第二章：凡人的极限，我的起点

- 情节：林默回家修炼基础吐纳法，平时修炼毫无进展，今日却如长江大河般吸纳灵气。一夜间，气血值飙升至100，突破至武者一品。但他选择隐藏实力，仅在暗中观察。
- 关键点：首次体验“万倍返还”的快感。

第三章：赵公子的刁难

- 情节：班级实战课，赵天宇故意挑衅，要求与林默切磋。老师无奈同意。林默“勉强”接下三招，赵天宇欲下重手，被林默“恰好”躲过，反而摔了个狗吃屎。
- 冲突：赵天宇怀恨在心，暗中联系地下黑拳组织。

第四章：地下拳场的赌注



【实验总结】

迭代的重要性： 我深刻体会到，好的提示词不是“写”出来的，而是“调”出来的。 在实验中，初版提示词往往无法一次性达到完美效果，通过分析输出中的“信息遗漏”和“逻辑硬伤”，我学会了如何针对性地增加约束条件或调整指令顺序。

闭环流程的建立： 我成功建立了一套“设计—测试—分析—修正”的闭环流程。这种工程化的思维模式，让我在面对不理想的输出时不再盲目修改，而是有据可依地进行优化。

实战能力提升： 经过这三次实验，我已经能够独立设计高效的提示词，并能利用 AI 工具更高效地辅助我的学习与工作，这为我未来处理更复杂的 AI 项目打下了坚实的基础。